Universidade Federal de Mato Grosso
Instituto de Ciências Exatas e da Terra
Campus Araguaia

Trabalho 2

Relógio Digital em RISC-V Bare Metal

com UART e Interrupções

Relatório Técnico e Explicação de Código

Alunas:	Raissa Cavalcanti e Vitória Calonga
Disciplina:	Organização de Computadores
Professor:	Linder Cândido
Data:	Março de 2025

Sumário

1. Visão Geral do Projeto
2. Arquitetura de Hardware Emulado
3. Script de Ligação - linker.ld
4. Inicialização e Trap Entry - start.S
5. Despachador de Interrupções - trap.c
6. Driver do Temporizador - timer.c
7. Driver UART - uart.c
8. Driver PLIC - plic.c
9. Programa Principal - main.c
10. Habilitação de Interrupções
11. Fluxo Completo de Execução
12. Conclusão

1. Visão Geral do Projeto

Este trabalho implementa um relógio digital bare metal para a arquitetura RISC-V 64 bits, executando diretamente sobre o hardware emulado pelo QEMU (máquina virt), sem sistema operacional, sem bibliotecas C padrão e sem qualquer rotina externa.

O projeto demonstra conceitos fundamentais de organização de computadores:

- Mapeamento de periféricos em memória (memory-mapped I/O)
- Interrupções de hardware: temporizador (CLINT) e externas via PLIC
- Comunicação serial via UART NS16550A
- Programação direta de CSRs (Control and Status Registers) do RISC-V
- Salvamento e restauração de contexto em rotinas de tratamento de interrupção

Estrutura de arquivos do projeto:

Arquivo	Linguagem	Função
start.S	Assembly	Inicialização + stub de entrada de trap
main.c	C	Loop principal, exibição da hora, parse de comando
uart.c	C	Driver do UART NS16550A (TX polling, RX por interrupção)
timer.c	C	Driver do CLINT e lógica de contagem do relógio
trap.c	C	Despachador geral de interrupções
plic.c	C	Driver do PLIC (roteamento de IRQ externo)
types.h	C (header)	Tipos básicos e macros de CSR
linker.ld	Linker	Layout de memória

2. Arquitetura de Hardware Emulado

O QEMU emula a máquina virt, uma plataforma de referência RISC-V com periféricos acessados por endereços físicos fixos (memory-mapped I/O). O código lida diretamente com esses endereços, sem nenhuma camada de abstração.

Periférico	Endereço Base	Uso no projeto
DRAM (RAM)	0x80000000	Código, dados, BSS e pilha do programa
UART NS16550A	0x10000000	Comunicação serial bidirecional com o terminal
CLINT (Timer)	0x02000000	Registradores mtime e mtimecmp para temporizador
PLIC	0x0C000000	Roteamento de IRQs externos (UART) para os harts

i *Todos os acessos a periféricos usam ponteiros volatile para garantir que o compilador não otimize nem reordene as leituras/escritas em registradores de hardware.*

3. Script de Ligação - linker.ld

O script de ligação define o layout de todas as seções do programa na memória física. Sem ele, o ligador não saberia onde posicionar o código para que o QEMU execute corretamente.

Linha a linha:

`OUTPUT_ARCH(riscv)`	Informa ao ligador que o binário de saída é para arquitetura RISC-V.
`ENTRY(_start)`	Define _start (em start.S) como ponto de entrada do programa - primeiro endereço executado.
`MEMORY { RAM (rwx) : ORIGIN = 0x80000000, LENGTH = 128M }`	Declara a região de memória disponível: 128 MB de RAM iniciando em 0x80000000, com permissões de leitura (r), escrita (w) e execução (x).
`. = 0x80000000;`	Posiciona o contador de localização no início da RAM, onde o QEMU carrega o kernel.
`*(.text.start)`	Garante que a seção especial .text.start (onde _start está definido) venha primeiro, ficando exatamente em 0x80000000.
`*(.text .text.*)`	Agrupa todo o restante do código executável logo após.
`.rodata`	Seção de dados somente-leitura: strings constantes, tabelas, etc.
`.data`	Variáveis globais inicializadas (com valor definido no código-fonte).
`_bss_start / _bss_end`	Marcadores do início e fim da seção BSS. O código em start.S usa esses símbolos para zerar a região no boot.
`_stack_top = . + 0x10000`	Reserva 64 KB para a pilha. _stack_top aponta para o topo, onde sp é inicializado (a pilha RISC-V cresce para baixo).

4. Inicialização e Trap Entry - start.S

Este arquivo assembly é o coração do boot bare metal. Tem duas responsabilidades: inicializar o ambiente antes de chamar main() e fornecer o ponto de entrada para toda interrupção ou exceção.

4.1 Rotina _start (Boot)

```
  --- _start ---
    la   sp, _stack_top
    # Carrega o endereço de _stack_top em sp.
    # _stack_top é definido no linker.ld como o topo da área de 64 KB de pilha.
    # Em RISC-V a pilha cresce para baixo, por isso começamos no endereço mais alto.

    la   t0, trap_entry
    csrw mtvec, t0
    # Instala o vetor de traps: mtvec recebe o endereço de trap_entry.
    # Com bits [1:0]=00 (modo Direto), qualquer interrupção salta para esse endereço.

    la   t0, _bss_start
    la   t1, _bss_end
.Lclear_bss:
    bge  t0, t1, .Ldone_bss    # se t0 >= t1, sair do loop
    sd   zero, 0(t0)           # escrever 8 bytes zero no endereço t0
    addi t0, t0, 8             # avançar 8 bytes (sd = store doubleword)
    j    .Lclear_bss           # repetir
    # Zera toda a seção BSS — variáveis globais devem começar com zero.

    call main       # transfere controle para o programa C principal
.Lhalt: j .Lhalt   # loop de segurança: main() nunca deve retornar
```

4.2 Rotina trap_entry (Stub de Salvamento de Contexto)

Quando qualquer interrupção ocorre, o hardware RISC-V automaticamente salva mepc (endereço de retorno) e mstatus, e salta para mtvec. O código assembly abaixo completa o salvamento:

```
  --- trap_entry ---
    addi sp, sp, -256
    # Reserva 256 bytes na pilha: 32 registradores × 8 bytes.
    # A pilha cresce para baixo, por isso subtrai.

    sd   ra,    0(sp)    # salva x1  (return address)
    sd   gp,   16(sp)    # salva x3  (global pointer)
    sd   t0,   32(sp)    # salva x5  (temporário)
    sd   a0,   72(sp)    # salva x10 (argumento/retorno)
    # ... (todos os 31 registradores de uso geral são salvos)
    # Por que salvar tudo? A ISR em C pode chamar funções que
    # modificam qualquer registrador. Sem salvar, o código
    # interrompido seria corrompido ao retornar.

    addi t0, sp, 256
    sd   t0,   8(sp)     # salva o valor original de sp (x2)

    call trap_handler    # chama o despachador C
```

```
    ld   ra,    0(sp)    # restaura todos os registradores
    # ... (processo inverso do salvamento)

    addi sp, sp, 256     # libera o frame da pilha
    mret                 # Machine Return:
    # Restaura mstatus e salta para mepc (ponto de interrupção).
```

5. Despachador de Interrupções - trap.c

Esta função em C é chamada pelo stub assembly. Ela lê o CSR mcause para identificar a origem do trap e encaminha para a rotina correta.

Como funciona o mcause:

- **O trap foi causado por uma interrupção assíncrona (hardware).** Bit 63 = 1:
- **O trap foi causado por uma exceção síncrona (instrução ilegal, acesso inválido, etc.).** Bit 63 = 0:
- **O código da causa específica. Causa 7 = Timer; Causa 11 = Externa (PLIC).** Bits 62–0:

```
--- void trap_handler(void) ---
  unsigned long mcause = read_csr(mcause);
  # Lê o CSR mcause via instrução 'csrr' (ver macros em types.h).

  int is_interrupt = (mcause >> 63) & 1;
  # Desloca 63 bits à direita e isola o bit 0 = bit 63 original.
  # Se 1: é interrupção. Se 0: é exceção síncrona.

  unsigned long cause = mcause & ~(1UL << 63);
  # Máscara: zera o bit 63, deixando apenas o código de causa.

  if (cause == IRQ_M_TIMER) {
      timer_interrupt_handler();   // 1 segundo passou
  } else if (cause == IRQ_M_EXT) {
      uint32_t source = plic_claim();     // quem gerou o IRQ?
      if (source == PLIC_UART0_IRQ)       // source 10 = UART0
          uart_interrupt_handler();
      plic_complete(source);  // OBRIGATÓRIO: confirma ao PLIC
  }
  # Sem plic_complete(), o PLIC nunca geraria nova IRQ da mesma fonte.
```

ℹ *O ciclo claim → tratar → complete é obrigatório para cada interrupção externa. O 'claim' atomicamente 'toma posse' da interrupção e o 'complete' libera a fonte para novas interrupções.*

6. Driver do Temporizador - timer.c

O CLINT (Core Local Interruptor) fornece a base de tempo do sistema. Dois registradores de 64 bits, mapeados em memória, controlam todo o mecanismo:

- **contador livre, incrementado pelo hardware a 10 MHz (1 incremento a cada 100 ns).** mtime (0x200BFF8):
- **quando mtime >= mtimecmp, o bit MTIP é setado, gerando interrupção.** mtimecmp (0x2004000):

```
--- Definições em timer.h ---
  #define CLINT_BASE      0x02000000UL
  #define CLINT_MTIME     (*(volatile uint64_t*)(CLINT_BASE + 0xBFF8UL))
  #define CLINT_MTIMECMP  (*(volatile uint64_t*)(CLINT_BASE + 0x4000UL))
  # Ponteiro volatile para uint64_t: acesso direto ao registrador de hardware.
  # 'volatile' impede que o compilador cache o valor em registrador de CPU.

  #define TIMER_FREQ_HZ  10000000UL  // 10 MHz = 10.000.000 ticks/segundo
```

Inicialização:

```
--- void timer_init(void) ---
  CLINT_MTIMECMP = CLINT_MTIME + TIMER_FREQ_HZ;
  # Lê o valor atual de mtime e soma 10.000.000 (= 1 segundo).
  # Quando mtime alcançar esse valor, a primeira interrupção será gerada.
```

ISR do Timer - Lógica Central do Relógio:

```
--- void timer_interrupt_handler(void) ---
  CLINT_MTIMECMP += TIMER_FREQ_HZ;
  # PASSO 1: Limpar a interrupção e agendar a próxima.
  # No RISC-V, a ÚNICA forma de limpar MTIP é escrever em mtimecmp.
  # Incrementar (em vez de reler mtime) evita deriva acumulada.

  clock_seconds++;
  if (clock_seconds >= 60) {
      clock_seconds = 0;
      clock_minutes++;
      if (clock_minutes >= 60) {
          clock_minutes = 0;
          clock_hours++;
          if (clock_hours >= 24)
              clock_hours = 0;   // retorno a 00:00:00 após 23:59:59
      }
  }
  # PASSO 2: Carry — segundos -> minutos -> horas.

  clock_updated = 1;   // sinaliza loop principal para redesenhar
  # PASSO 3: Não imprime diretamente na ISR — uart_putc usa polling
  # e poderia travar a interrupção por tempo demais.
```

7. Driver UART - uart.c

O NS16550A é um UART clássico com registradores de 8 bits acessados por offset a partir do endereço base 0x10000000.

Registradores principais do NS16550A:

Offset	Nome	DLAB	Uso
0	RBR/THR	0	Receive Buffer / Transmit Holding Register
1	IER	0	Interrupt Enable Register — habilita interrupções individuais
2	IIR/FCR	-	Interrupt ID Register (leitura) / FIFO Control (escrita)
3	LCR	-	Line Control Register — formato do frame (bits, paridade, stop)
5	LSR	-	Line Status Register — status TX (THRE) e RX (DR)
0/1	DLL/DLM	1	Divisor Latch LSB/MSB — configuração do baud rate

Inicialização:

```
--- void uart_init(void) ---
  UART_REG(UART_IER) = 0x00;    // desabilita IRQs durante configuração
  UART_REG(UART_LCR) = UART_LCR_DLAB;  // DLAB=1: acessa divisor baud
  UART_REG(UART_DLL) = 0x01;    // divisor LSB = 1
  UART_REG(UART_DLM) = 0x00;    // divisor MSB = 0
  # DLAB=1: offsets 0/1 acessam divisor em vez de RBR/THR e IER.
  UART_REG(UART_LCR) = UART_LCR_8N1;   // 8 bits, sem paridade, 1 stop
  UART_REG(UART_FCR) = 0x07;    // habilita FIFO + reset TX + reset RX
  UART_REG(UART_IER) = UART_IER_RDI;   // IRQ quando dado disponível (bit 0)
```

Transmissão por Polling:

```
--- void uart_putc(char c) ---
  while (!(UART_REG(UART_LSR) & UART_LSR_THRE));
  # Aguarda bit 5 (THRE) do LSR = 1:
  # Transmit Holding Register Empty — transmissor pronto.
  UART_REG(UART_THR) = (uint8_t)c;
  # Escreve o byte. O hardware envia automaticamente.
```

ISR de Recepção:

```
--- void uart_interrupt_handler(void) ---
  while (UART_REG(UART_LSR) & UART_LSR_DR) {  // DR = Data Ready
  # Loop enquanto há bytes no FIFO de recepção.
      char c = (char)UART_REG(UART_RBR);  // lê próximo byte

      if (c == '\r' || c == '\n') {
          uart_rx_buf[uart_rx_idx] = '\0'; // fecha string
          uart_cmd_ready = 1;              // sinaliza main()
      } else if (c == 0x7F || c == '\b') { // backspace
          if (uart_rx_idx > 0) uart_rx_idx--;
          uart_putc_raw('\b'); uart_putc_raw(' '); uart_putc_raw('\b');
      } else if (c >= 0x20 && c <= 0x7E) { // imprimível
          uart_rx_buf[uart_rx_idx++] = c;
```

```
            uart_putc_raw(c);  // eco: mostrar o que o usuário digitou
        }
        # Bytes de controle (< 0x20) descartados silenciosamente.
        # Evita 'comandos inválidos' espúrios do QEMU na inicialização.
    }
```

8. Driver PLIC - plic.c

O PLIC (Platform-Level Interrupt Controller) recebe sinais de vários periféricos (fontes) e os roteia para os contextos de execução (hart × modo). No QEMU virt, o UART0 é a fonte 10 e o contexto 0 corresponde ao hart 0 em modo Machine.

```
--- void plic_init(void) ---
  PLIC_PRIORITY(PLIC_UART0_IRQ) = 1;
  # Define prioridade 1 para a fonte 10 (UART0).
  # O PLIC ignora fontes com prioridade 0 (desabilitadas).

  PLIC_ENABLE_CTX0 = (1U << PLIC_UART0_IRQ);  // bit 10 = fonte 10
  # Registrador de 32 bits: bit n = 1 habilita fonte n no contexto 0.

  PLIC_THRESHOLD_CTX0 = 0;
  # Threshold = 0: aceitar interrupções de qualquer prioridade >= 1.

--- uint32_t plic_claim(void) ---
  return PLIC_CLAIM_CTX0;
  # Retorna o ID da fonte pendente de maior prioridade.
  # Atomicamente 'toma posse': impede nova IRQ da mesma fonte
  # até que plic_complete() seja chamado.

--- void plic_complete(uint32_t source) ---
  PLIC_CLAIM_CTX0 = source;
  # Sinaliza que o tratamento terminou.
  # Sem isso, o PLIC não geraria novos IRQs do UART.
```

9. Programa Principal - main.c

9.1 Função main()

```
--- int main(void) ---
  uart_init();    // configura NS16550A
  timer_init();   // agenda primeira interrupção de timer
  plic_init();    // configura prioridade e enable do UART no PLIC
  # Ordem importa: periféricos devem estar prontos ANTES das IRQs.

  write_csr(mie, MIE_MTIE | MIE_MEIE);
  # Habilita no CSR mie:
  #   MIE_MTIE (bit 7)  = Machine Timer Interrupt Enable
  #   MIE_MEIE (bit 11) = Machine External Interrupt Enable

  set_csr(mstatus, MSTATUS_MIE);
  # Liga a chave global de interrupções (mstatus.MIE = bit 3).
  # Sem isso, nenhuma interrupção seria atendida.

  while (1) {
      if (clock_updated) {      // ISR do timer levantou esta flag?
          clock_updated = 0;
          display_time();       // imprime HH:MM:SS no terminal
      }
      if (uart_cmd_ready) {     // ISR do UART recebeu comando?
          parse_command();      // interpreta 'T HH:MM:SS'
          display_time();       // redesenha imediatamente
      }
  }  // CPU fica aqui; toda a lógica vem das ISRs
```

9.2 Exibição da Hora

```
--- static void display_time(void) ---
  uart_puts('\r[');              // \r: volta ao início da mesma linha
  uart_put2digit(clock_hours);    uart_putc(':');
  uart_put2digit(clock_minutes);  uart_putc(':');
  uart_put2digit(clock_seconds);
  uart_puts('] > ');              // prompt para digitação
  # \r sem \n sobrescreve a linha → efeito de relógio ao vivo.
  # uart_put2digit(n): imprime com 2 dígitos. Ex: n=5 -> '05'.
```

9.3 Interpretação do Comando T HH:MM:SS

```
--- static void parse_command(void) ---
  if (i < 10)                          goto cmd_error; // mínimo 10 chars
  if (buf[0]!='T' || buf[1]!=' ')       goto cmd_error;
  if (!is_digit(buf[2])||!is_digit(buf[3])) goto cmd_error;
  if (buf[4] != ':')                    goto cmd_error;
  // ... validação de minutos e segundos ...

  int h = (buf[2]-'0')*10 + (buf[3]-'0');  // '1','4' -> 14
  if (h > 23 || m > 59 || s > 59) goto cmd_error;

  clear_csr(mstatus, MSTATUS_MIE);  // desabilita IRQs globalmente
  clock_hours = h; clock_minutes = m; clock_seconds = s;
```

```
    set_csr(mstatus, MSTATUS_MIE);    // reabilita IRQs
    # Seção crítica: desabilitar IRQs evita leitura de estado parcial
    # pela ISR do timer durante a atualização dos contadores.
```

10. Habilitação de Interrupções no RISC-V

Para que uma interrupção seja atendida, três níveis de controle devem estar habilitados simultaneamente:

Nível	Registrador	Bit / Campo	Descrição
1 - Global	CSR mstatus	Bit 3 (MIE)	Chave mestre: habilita/desabilita TODAS as interrupções em M-mode
2 - Por tipo	CSR mie	Bit 7 (MTIE) / Bit 11 (MEIE)	Habilita timer ou externas individualmente
3 - Por fonte	PLIC / CLINT	Enable bits / mtimecmp	Para externas: habilita fonte no PLIC. Para timer: mtimecmp > mtime atual

Macros de CSR em types.h:

```
--- Macros de acesso a CSRs ---
  #define read_csr(reg)  ({ unsigned long v; __asm__("csrr %0,reg" : "=r"(v)); v; })
  # csrr: instrução RISC-V 'CSR Read' — lê o registrador especificado.

  #define write_csr(reg, val)  __asm__("csrw reg, %0" :: "r"(val))
  # csrw: 'CSR Write' — escreve valor no registrador.

  #define set_csr(reg, bit)    // csrrs: CSR Read and Set bits
  #define clear_csr(reg, bit)  // csrrc: CSR Read and Clear bits
  # csrrs/csrrc são atômicas: leem e modificam bits em uma instrução.
  # Essencial para seções críticas sem race condition.
```

11. Fluxo Completo de Execução

11.1 Boot (uma única vez)

1. QEMU carrega clock.elf em 0x80000000 e salta para _start
2. sp ← _stack_top (topo dos 64 KB de pilha)
3. mtvec ← endereço de trap_entry (modo direto)
4. BSS zerada iterando de _bss_start até _bss_end
5. main() chamado; periféricos inicializados: uart_init() → timer_init() → plic_init()
6. mie habilitado: MTIE + MEIE; mstatus.MIE habilitado
7. Loop while(1) iniciado — CPU aguarda flags

11.2 A cada 1 segundo (interrupção de timer)

1. CLINT: mtime atinge mtimecmp → MTIP setado
2. CPU: completa instrução atual, salva mepc/mstatus, salta para trap_entry
3. trap_entry: salva 31 registradores na pilha
4. trap_handler(): mcause = 0x8000000000000007 → causa 7 → timer_interrupt_handler()
5. mtimecmp += 10.000.000 (limpa MTIP + agenda próximo segundo)
6. Incrementa clock_seconds/minutes/hours com carry; clock_updated = 1
7. mret → loop principal: clock_updated==1 → display_time()

11.3 Ao receber um caractere via UART

1. UART NS16550A: byte chega no FIFO → IRQ sinalizada ao PLIC
2. PLIC: verifica prioridade/threshold → sinaliza MEIP ao hart 0
3. CPU: salta para trap_entry (mesmo mecanismo do timer)
4. trap_handler(): mcause = 0x800000000000000B → causa 11 → IRQ externa
5. plic_claim() → retorna 10 (source UART0) → uart_interrupt_handler()
6. Drena FIFO, armazena bytes, eco no terminal; se '\r': uart_cmd_ready = 1
7. plic_complete(10) → PLIC libera para novos IRQs
8. Loop principal: uart_cmd_ready==1 → parse_command() → atualiza relógio

12. Conclusão

O projeto implementa com sucesso todos os requisitos propostos, demonstrando na prática os conceitos centrais de organização de computadores:

- **Programação bare metal:** nenhuma camada de abstração entre o software e o hardware — cada registrador é acessado diretamente pelo endereço físico.
- **Interrupções como mecanismo central:** tanto a contagem de tempo quanto a recepção de dados da UART são tratadas exclusivamente por interrupções, sem polling de tempo.
- **Mecanismo de trap do RISC-V:** o hardware salva apenas mepc e mstatus; o software (trap_entry) é responsável por salvar e restaurar o restante do contexto.
- **Hierarquia de habilitação:** mstatus.MIE (global) → mie (por tipo) → PLIC/CLINT (por fonte), todos os três níveis necessários.
- **Comunicação serial NS16550A:** inicialização com divisor de baud, framing 8N1, FIFO habilitado, IRQ de recepção — protocolo completo implementado manualmente.
- **Seções críticas:** desabilitação temporária de interrupções ao atualizar múltiplos campos do relógio para garantir consistência de dados.

O resultado é um sistema funcional que exibe a hora em tempo real via UART e aceita comandos de ajuste no formato T HH:MM:SS, executando inteiramente em modo Machine (M-mode) do RISC-V, sem qualquer dependência de sistema operacional ou biblioteca externa.